Evidencia de desempeño: GA4-220501095-AA2-EV05 - Desarrollar la arquitectura de software de acuerdo al patrón de diseño seleccionado

Presentado por:

Luis Alejandro Muñoz Sierra

Presentado a:

Naín Zúñiga

Servicio Nacional de Aprendizaje – SENA

Programa de Formación:

Análisis y desarrollo de software

Ficha: 2977356

Medellín, 30 de julio de 2025

Desarrollar la arquitectura de software de acuerdo al patrón de diseño seleccionado GA4-220501095-AA2-EV05

Software de Gestión de Maquinaria Pesada

Introducción

La arquitectura de software es el pilar sobre el cual se construye un sistema informático de calidad. No se trata solo de la tecnología, sino de cómo se estructuran sus partes, cómo interactúan y cómo evoluciona en el tiempo. Una arquitectura bien definida garantiza que un sistema sea robusto, mantenible y escalable.

Este documento detalla la arquitectura de software diseñada para el proyecto "MaquiMu". Se presenta el patrón de arquitectura seleccionado, la Arquitectura Hexagonal, junto con patrones de diseño específicos que promueven las mejores prácticas. Además, se incluyen las vistas de componentes y de despliegue para ofrecer una visión completa tanto de la estructura lógica como de la implementación física del sistema, y se describen las herramientas clave que optimizarán el ciclo de vida del desarrollo.

1. Patrón de Arquitectura: Arquitectura Hexagonal (Puertos y Adaptadores)

Para el proyecto "MaquiMu", se ha seleccionado la Arquitectura Hexagonal, también conocida como patrón de Puertos y Adaptadores. Este patrón es ideal para crear aplicaciones desacopladas, donde la lógica de negocio central es independiente de las tecnologías externas como la interfaz de usuario, la base de datos o servicios de terceros.

Imagina la lógica de negocio como el núcleo (el hexágono). Este núcleo no sabe ni le importa si es controlado por un usuario a través de una página web, una aplicación móvil o una línea de comandos. Tampoco sabe si sus datos se guardan en una base de datos MySQL, PostgreSQL o un archivo de texto.

La comunicación entre el núcleo y el mundo exterior se realiza a través de "Puertos" (que son simplemente interfaces en el código) y "Adaptadores" (que son las implementaciones concretas de esas interfaces).

- **El Núcleo (Dominio y Aplicación):** Contiene las clases de entidad (Cliente, Maquinaria, etc.) y los casos de uso (AlquilerService). Es la parte más importante y no tiene dependencias externas.

- **Puertos (Ports):** Son las interfaces definidas por el núcleo que exponen su funcionalidad. Por ejemplo, un puerto AlquilerRepositoryPort define métodos como guardar(alquiler) o buscarPorId(id).

- **Adaptadores (Adapters):** Son las clases que "traducen" la comunicación. Un controlador de la API REST es un adaptador que recibe una petición HTTP y llama a un puerto del núcleo. Una clase que interactúa con MySQL es un adaptador que implementa la interfaz AlquilerRepositoryPort.

La elección de este patrón para "MaquiMu" se justifica porque facilita las pruebas (se puede probar el núcleo sin necesidad de una base de datos o una interfaz gráfica), mejora la mantenibilidad y permite cambiar tecnologías externas (como migrar de MySQL a otra base de datos) con un impacto mínimo en la lógica de negocio.

2. Incorporación de Patrones de Diseño de Software

Dentro de la Arquitectura Hexagonal, utilizaremos patrones de diseño específicos para resolver problemas comunes y seguir las mejores prácticas:

- **Patrón Repositorio (Repository Pattern):** Este patrón mediará entre el dominio y la capa de mapeo de datos. Crearemos interfaces como `ClienteRepository` o `MaquinariaRepository` (los "puertos") que serán implementadas por clases concretas que usan Spring Data JPA para interactuar con MySQL. Esto abstrae completamente el acceso a los datos.
- **Patrón Capa de Servicio (Service Layer Pattern):** Los casos de uso se implementarán en clases de servicio como `AlquilerService`. Esta capa actuará como una fachada (`Facade`) para la lógica de negocio, coordinando las operaciones con los repositorios y otros servicios, manteniendo la lógica limpia y organizada.
- **Patrón DTO (Data Transfer Object):** Para la comunicación a través de la API REST, usaremos objetos DTO. Por ejemplo, al crear un cliente, el frontend enviará un `ClienteCreacionDTO` en lugar de exponer

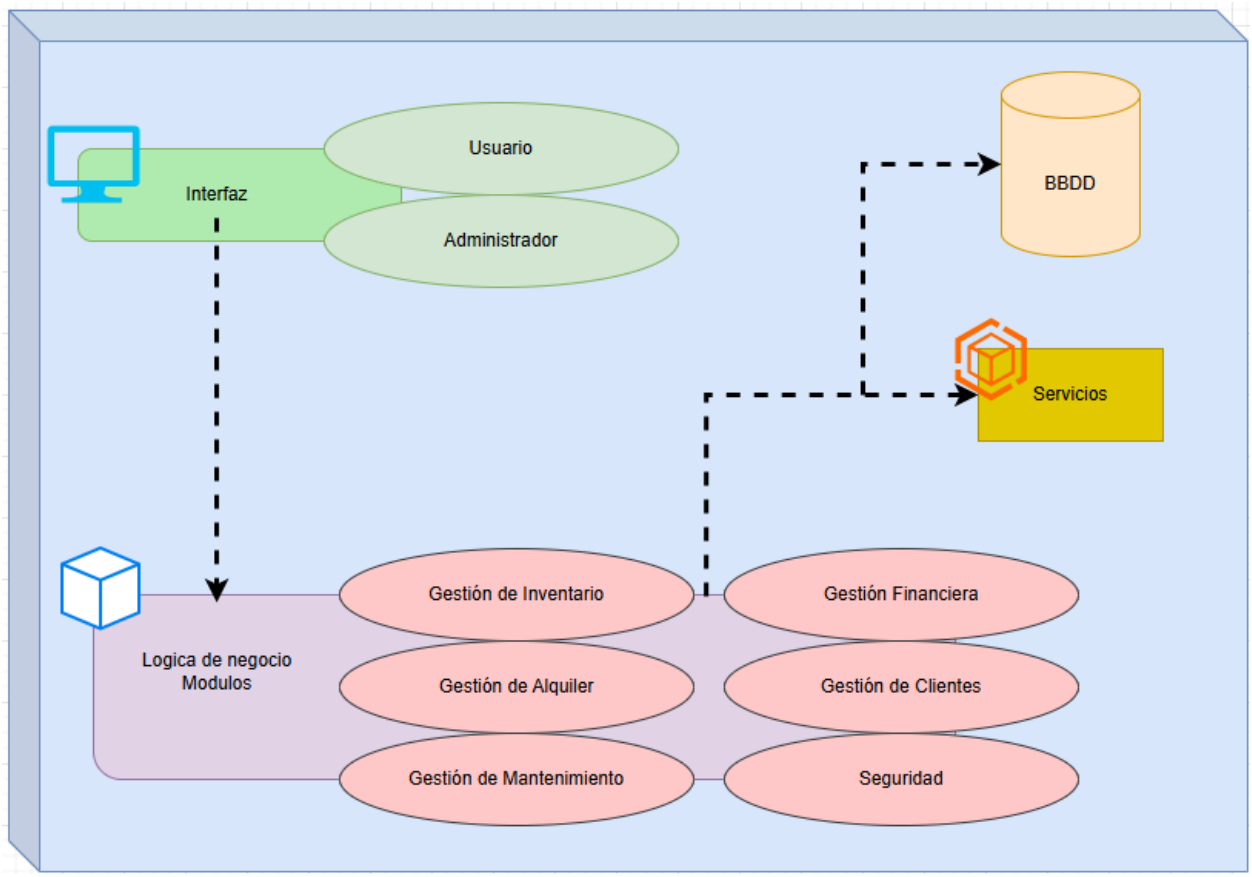
directamente el objeto de dominio `Cliente`. Esto mejora la seguridad y desacopla la vista de la lógica interna.

3. Vista de Componentes

La vista de componentes muestra cómo el sistema se divide en componentes lógicos y las dependencias entre ellos. Para "MaquiMu", los principales componentes son:

1. **Componente Web (Frontend):** Es la aplicación de una sola página (SPA) desarrollada en **Angular**. Se encarga de toda la interacción con el usuario.
2. **Componente API REST:** Es el punto de entrada al backend. Expone los servicios del sistema a través de *endpoints* HTTP y está implementado con **Spring Boot**.
3. **Componente Núcleo de Negocio:** Contiene toda la lógica de negocio pura (servicios y entidades del dominio). No depende de ningún otro componente, son los otros los que dependen de él.
4. **Componente de Persistencia:** Implementa los puertos del repositorio para interactuar con la base de datos. Utiliza Spring Data JPA para mapear objetos a la base de datos MySQL.

El flujo es el siguiente: El **Componente Web** se comunica con el **Componente API REST**. Este, a su vez, utiliza el **Componente Núcleo de Negocio** para ejecutar las operaciones, y el núcleo usa el **Componente de Persistencia** para guardar o recuperar datos.

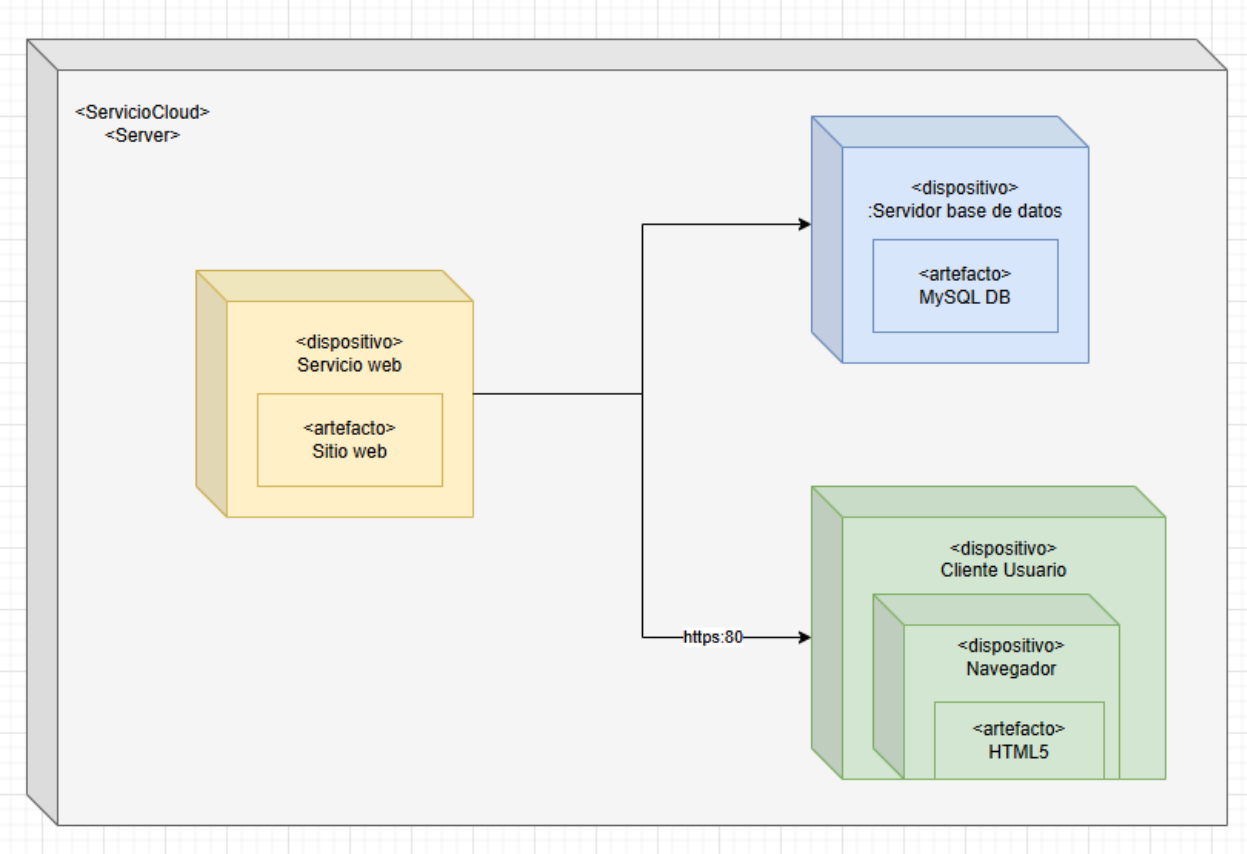


4. Vista de Despliegue

La vista de despliegue muestra cómo los componentes de software se distribuyen en la infraestructura física (o virtualizada) en producción. Para "MaquiMu", siendo una aplicación nativa de la nube, el despliegue se realizará en **Amazon Web Services (AWS)**.

1. **Nodo Cliente:** El navegador web del usuario en su computadora o dispositivo móvil.

2. **Nodo de Distribución de Contenido (CDN):** Se usará **Amazon S3** para alojar los archivos estáticos de la aplicación Angular y **Amazon CloudFront** como CDN para distribuirlos globalmente con baja latencia.
3. **Nodo de Aplicación (Contenedor):** El backend de **Spring Boot** se empaquetará en un contenedor **Docker** y se ejecutará en un servicio de orquestación como **Amazon ECS (Elastic Container Service)**. Esto permite escalar la aplicación fácilmente si la carga aumenta.
4. **Nodo de Base de Datos:** Se utilizará **Amazon RDS (Relational Database Service)** para desplegar una instancia gestionada de **MySQL**, lo que simplifica las copias de seguridad, el mantenimiento y la alta disponibilidad.



5. Herramientas para la Optimización de Procesos

Para asegurar un ciclo de desarrollo eficiente y de alta calidad, se utilizarán las siguientes herramientas:

- **Gestión de Proyecto: Jira**, para planificar y dar seguimiento a las tareas de cada fase del ciclo de vida en cascada.
- **Control de Versiones: Git**, con repositorios alojados en **GitHub**, para gestionar el código fuente de manera colaborativa.
- **Integración y Entrega Continua (CI/CD): Jenkins** o **GitLab CI**, para automatizar los procesos de compilación, pruebas y despliegue cada vez que se realiza un cambio en el código.
- **Calidad de Código: SonarQube**, para realizar análisis estático del código, detectar bugs, vulnerabilidades y "code smells", garantizando que se sigan las mejores prácticas.
- **Contenerización: Docker**, para empaquetar la aplicación y sus dependencias, asegurando que se ejecute de la misma manera en cualquier entorno.

Conclusiones

La arquitectura de software definida para "MaquiMu" proporciona una estructura sólida, moderna y alineada con las mejores prácticas de la industria. La elección de la Arquitectura Hexagonal, complementada con patrones de diseño como Repository y DTO, asegura un sistema desacoplado, fácil de probar y mantener.

Las vistas de componentes y despliegue clarifican tanto la organización lógica del software como su implementación en una infraestructura de nube escalable y resiliente. Finalmente, el uso de herramientas de automatización y calidad optimizará el proceso de desarrollo, resultando en un producto final robusto y confiable.

HISTORIAL DE MODIFICACIONES

Versión	Naturaleza de cambio	fecha de aprobación	Fecha validación
1.0	Resumen arquitectura	2025-07-29	2025-07-30

FIRMAS

Elaboró	Aprobó	Validó
Luis Alejandro Muñoz SIerra	Inversionistas S.A.	Secretaria MaquiMu
Secretaria MaquiMu	Inversionistas S.A.	Gerente de turno.